

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 4 Laboratory Sheet

Search II: Local Search, Restarts and Learning

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours
Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying ortools), a text editor

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters

Week 3's backtracking search chokes the moment a problem's search tree grows too large to build at all, while CP-SAT proves the order-9 Golomb ruler's optimal length in well under a second. This lab hands you the four techniques behind that gap: starting from a complete guess and repairing it one variable at a time, handing a whole relaxed chunk of variables to CP-SAT itself to repair, never rediscovering the exact same search failure twice, and running several of the above at once on separate CPU cores. By the end of the next two hours you will have watched all four mechanisms run on real code, and closed the sheet by combining two of them yourself on an instance harder than any single reference script here solves comfortably.

1 • Pre-instruction Prior Knowledge

This sheet exercises all four units of this week's chapter: local search and the min-conflicts heuristic (Unit 1), large neighbourhood search (Unit 2), restarts with nogood learning and lazy clause generation (Unit 3), and portfolio/parallel search (Unit 4). It builds directly on Week 3's backtracking search and, in particular, the `num_search_workers` parameter you already set there without being told what it actually launches — Unit 4 below is where that gap finally closes. Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Confirming the Shared Environment

Step 1: Confirm this week needs no new package

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` active, you will print the installed `ortools` version to confirm the same package Week 1 installed and Week 3 already used is still there — this week is CP-SAT only, no new package

Step 1 (continued)

install is needed.

Why? Every real solver run and captured log in this sheet depends on the exact `ortools` version this course is pinned to; confirming it once, before you start, is what makes every `ciktikutusu` box below reproducible on your own machine.

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ cd hafta04
$ python -c "import ortools; print(ortools.__version__)"
```

Expected result: The command prints `9.14.6206`, the exact version Week 1 pinned for the whole course.

Expected Output

```
9.14.6206
```

Tip

If this command raises `ModuleNotFoundError`, your virtual environment is not active in this terminal — re-run `source .venv/bin/activate` from inside `kod_ornekleri`. Every remaining command in this sheet is run from inside `kod_ornekleri/hafta04`.

2 Unit 1 — Local Search for CSPs

(See Course Notes Week 4, Unit 1 — Local Search for CSPs.)

Step 2: Run min_conflicts.py: watch min-conflicts repair 2000 queens

What will you do? You will run `modeller/min_conflicts.py`, a pure Python (no CP-SAT) min-conflicts local search over `N=2000` queens started from a random complete assignment, and read the printed step count and final conflict count.

Why? This is Unit 1's own min-conflicts heuristic, running for real at a scale no hand trace could reach: at each step it picks a row still involved in a violated constraint and reassigns it to the column that violates the fewest of the others — exactly the single-move rule the chapter's own 4-queens practice task asks you to hand-compute, now watched at 2000 queens instead of 4.

```
col_count[columns[row]] -= 1
diag1_count[row - columns[row] + n - 1] -= 1
diag2_count[row + columns[row]] -= 1

best_columns, best_conflicts = [], n + 1
for col in range(n):
    conflicts = col_count[col] + diag1_count[row - col + n - 1] + diag2_count[row +
        col]
    if conflicts < best_conflicts:
        best_columns, best_conflicts = [col], conflicts
    elif conflicts == best_conflicts:
```

Step 2 (continued)

```
      best_columns.append(col)
new_col = rng.choice(best_columns)

columns[row] = new_col
col_count[new_col] += 1
diag1_count[row - new_col + n - 1] += 1
diag2_count[row + new_col] += 1
```

Terminal

```
$ python modeller/min_conflicts.py
```

Expected result: The search reaches zero conflicts and reports the real step count it took — not a fixed number known in advance, since every step picks among tied best columns at random.

Expected Output

```
Solved N=2000: 0 conflicts remaining after 1288 steps.
```

Tip

Each step recomputes conflict counts using the row/diagonal occupancy counters built in `min_conflicts_state`, not a full rescan of all 2000 queens — this is what keeps a 2000-queen run finishing in well under a second on ordinary hardware.

Step 3: Compare a second run with a different random seed

What will you do? Without editing `modeller/min_conflicts.py`, call `min_conflicts_loop` directly from the command line with the same `N` and step budget but seed 2 instead of the file's own seed 1, and compare the printed step count against the previous step's 1288.

Why? Min-conflicts breaks ties among equally-good columns at random (Unit 1); running the exact same algorithm on the exact same instance size with a different seed is what makes that randomness, and its effect on how many steps the repair takes, concrete rather than a line in the chapter you take on faith.

Terminal

```
$ python -c "import sys; sys.path.insert(0, 'modeller'); \
import min_conflicts as m; m.min_conflicts_loop(2000, 2, 100000)"
```

Expected result: The search again reaches zero conflicts, but after a different real step count than the previous step's run.

Expected Output

```
Solved N=2000: 0 conflicts remaining after 1309 steps.
```

Step 3 (continued)

Common Pitfall

Expecting the exact same step count as the previous step's run (1288) is a natural but wrong assumption — min-conflicts is not deterministic the way a systematic search with a fixed variable order is; only the seed, not the step count, is something you control directly.

☑ Checkpoint — verify before moving on

Both of your min-conflicts runs reached zero conflicts on 2000 queens, in 1288 and 1309 steps respectively — the same repair rule, the same instance size, two different real step counts because of two different random seeds. You have now watched Unit 1's heuristic run at a scale no hand trace could reach; Unit 2 next generalizes its one-variable repair step to a whole relaxed neighbourhood repaired by CP-SAT itself.

3 Unit 2 — Large Neighbourhood Search

(*See Course Notes Week 4, Unit 2 — Large Neighbourhood Search.*)

Step 4: Run large_nqueens_lns.py: destroy/repair on 50 queens

What will you do? You will run `modeller/large_nqueens_lns.py`, which starts from a random complete assignment for `N=50` queens and repeats, for 30 rounds, Unit 2's destroy/repair loop: relax `k=8` random rows, then hand the freed sub-problem to CP-SAT to re-solve against the 42 rows still fixed.

Why? Unlike the previous section's one-row-at-a-time min-conflicts move, each round here lets CP-SAT find the best possible placement for a whole relaxed neighbourhood at once, respecting every constraint that touches it — Unit 2's own generalization of Unit 1's repair habit, now watched converge round by round on a real run.

```
relaxed_rows = rng.sample(range(n), k)
```

```
model.AddAllDifferent([relaxed_vars[r] for r in relaxed_rows])
model.AddAllDifferent([relaxed_vars[r] + r for r in relaxed_rows])
model.AddAllDifferent([relaxed_vars[r] - r for r in relaxed_rows])
for row in relaxed_rows:
   for fixed_row in fixed_rows:
      fixed_col = columns[fixed_row]
      model.Add(relaxed_vars[row] != fixed_col)
      model.Add(relaxed_vars[row] + row != fixed_col + fixed_row)
      model.Add(relaxed_vars[row] - row != fixed_col - fixed_row)
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
   return columns
proposal = list(columns)
for row in relaxed_rows:
   proposal[row] = solver.Value(relaxed_vars[row])
return proposal if count_conflicts(proposal) <= count_conflicts(columns) else columns
```

Step 4 (continued)

Terminal

```
$ python modeller/large_nqueens_lns.py
```

Expected result: The printed conflict count drops from the initial random assignment's 64 conflicts toward zero over the 30 rounds, reaching zero by round 29 and staying there.

Expected Output

```
Round 0: 64 conflicts
Round 1: 50 conflicts
Round 2: 29 conflicts
Round 3: 25 conflicts
Round 4: 16 conflicts
Round 5: 7 conflicts
Round 6: 5 conflicts
Round 7: 5 conflicts
Round 8: 3 conflicts
Round 9: 2 conflicts
Round 10: 2 conflicts
Round 11: 2 conflicts
Round 12: 2 conflicts
Round 13: 2 conflicts
Round 14: 2 conflicts
Round 15: 1 conflicts
Round 16: 1 conflicts
Round 17: 1 conflicts
Round 18: 1 conflicts
Round 19: 1 conflicts
Round 20: 1 conflicts
Round 21: 1 conflicts
Round 22: 1 conflicts
Round 23: 1 conflicts
Round 24: 1 conflicts
Round 25: 1 conflicts
Round 26: 1 conflicts
Round 27: 1 conflicts
Round 28: 1 conflicts
Round 29: 0 conflicts
Round 30: 0 conflicts
```

Tip

Rounds where the count does not move (for example rounds 9–14, all at 2) are not wasted — that round's random neighbourhood did not happen to include a row that could improve the count. `lns_repair` always keeps the previous assignment whenever the repaired one is not at least as good, so a plateau is expected, not a bug.

Step 5: Run golomb_ruler.py: a proven optimum, then LNS-improve a bad start

What will you do? You will run `modeller/golomb_ruler.py`, which first solves the order-8 optimal Golomb ruler (OGR) to proven optimality, then builds a deliberately bad ruler with a greedy construction and shrinks it round by round with the same destroy/repair

Step 5 (continued)

loop as the previous step, relaxing `k=3` marks at a time for 15 rounds.

Why? This is Unit 2's second worked instance and its first genuine minimization objective, not a violated-constraint count: the base solve gives you a real, provably optimal target length to compare the LNS-improve run against, closing the loop between "CP-SAT can prove optimality outright" and "CP-SAT can also improve a bad starting point round by round."

```
distances = []
for i in range(len(marks)):
   for j in range(i + 1, len(marks)):
      distance = model.NewIntVar(1, upper_bound, f"dist_{i}_{j}")
      model.Add(distance == marks[j] - marks[i])
      distances.append(distance)
model.AddAllDifferent(distances)
```

```
for round_number in range(1, rounds + 1):
   relaxed = rng.sample(range(1, order), neighbourhood_size)
   proposal = _repair_relaxed_marks(marks, relaxed, order, upper_bound)
   if proposal is not None and _ruler_length(proposal) <= _ruler_length(marks):
      marks = proposal
   print(f"Round {round_number}: length {_ruler_length(marks)}, marks {marks}")
```

Terminal

```
$ python modeller/golomb_ruler.py
```

Expected result: The base solve reports status `OPTIMAL` and a proven length of 34 for 8 marks; the LNS-improve run then starts from a greedy ruler of length 44 and shrinks it to 41 by round 3, holding there through round 15.

Expected Output

```
Status: OPTIMAL
Optimal length for 8 marks: 34
Marks: [0, 1, 4, 9, 15, 22, 32, 34]
```

Expected Output

```
Round 0: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 1: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 2: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 3: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
Round 4: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
Round 5: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
Round 6: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
Round 7: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
Round 8: length 41, marks [0, 1, 9, 13, 15, 20, 38, 41]
Round 9: length 41, marks [0, 1, 9, 13, 15, 31, 38, 41]
Round 10: length 41, marks [0, 1, 9, 13, 15, 31, 34, 41]
Round 11: length 41, marks [0, 1, 9, 13, 15, 31, 34, 41]
Round 12: length 41, marks [0, 1, 9, 13, 15, 31, 34, 41]
Round 13: length 41, marks [0, 1, 9, 13, 15, 31, 38, 41]
Round 14: length 41, marks [0, 1, 11, 13, 18, 32, 38, 41]
Round 15: length 41, marks [0, 1, 11, 13, 18, 32, 38, 41]
```

Step 5 (continued)

> **Caution**
>
> `NEIGHBOURHOOD_SIZE` is set to 3, not 2, for a real reason: an exhaustive check over every 2-mark subset of this specific greedy start showed none of them can improve the length in a single round — the six marks left fixed already pin the pairwise distances too tightly. This is not a placeholder value; do not "simplify" it back to 2 while completing this week's capstone step.

Checkpoint — verify before moving on

`large_nqueens_lns.py` drove 50 queens from 64 conflicts to zero using CP-SAT as the repair engine, and `golomb_ruler.py` both proved an 8-mark ruler's optimum (34) and shrank a bad 44-length greedy start to 41 with the same destroy/repair loop. Unit 3 now asks what happens *inside* CP-SAT's own search every time one of these repair calls hits a dead end.

4 Unit 3 — Restarts and Nogood Learning

(*See Course Notes Week 4, Unit 3 — Restarts and Nogood Learning; Lazy Clause Generation.*)

Step 6: Run solver_log_restarts.py and locate the real restart/conflict counts

What will you do? You will run `modeller/solver_log_restarts.py`, which solves the order-9 Golomb-ruler optimality proof with `log_search_progress = True`, a single worker and a fixed random seed, saving CP-SAT's own verbose search log; you will then `grep` that log for the two summary lines reporting how many restarts and search failures ("conflicts," in CP-SAT's own sense) the proof needed.

Why? This is Unit 3's central claim made concrete with real evidence: every restart in this log is followed by newly learned nogoods pruning that combination away for good, rather than starting over from nothing — exactly the mechanism Unit 3 names as a large part of why CP-SAT is fast.

```
solver = cp_model.CpSolver()
solver.parameters.log_search_progress = True
solver.parameters.max_time_in_seconds = time_limit
solver.parameters.random_seed = seed
solver.parameters.num_search_workers = 1
status = solver.Solve(model)
print(f"Status: {solver.StatusName(status)}")
print(f"Length for {order} marks: {solver.Value(marks[-1])}")
```

```
Terminal

$ python modeller/solver_log_restarts.py > \
    ciktilar/solver-log-restarts.txt
$ grep -E "^conflicts:|^restarts:" \
    ciktilar/solver-log-restarts.txt
```

Expected result: Two lines: the total number of search failures ("conflicts") CP-SAT's own SAT-style engine recorded while proving the ruler's optimal length of 44, and the number of restarts it took along the way.

Step 6 (continued)

Expected Output

```
conflicts: 8573
restarts: 1139
```

Common Pitfall

"Conflicts" means two different things in this sheet. The previous section's board "conflicts" count attacking queen pairs in a complete assignment (Units 1–2). The `conflicts: 8573` line above is a completely different quantity, on a completely different problem: CP-SAT's own internal count of search failures its SAT-style engine turned into a nogood while proving the order-9 Golomb ruler's optimum. Never compare the two numbers to each other.

Checkpoint — verify before moving on

Your own real captured log shows 1139 restarts and 8573 recorded search failures behind a single `OPTIMAL` proof — each one of those 1139 restarts began knowing more than the last, because of the nogoods recorded along the way. Unit 4 next asks what changes when several searches like this one run at once.

5 Unit 4 — Portfolios and Parallel Search

(*See Course Notes Week 4, Unit 4 — Portfolios and Parallel Search.*)

Step 7: Run portfolio_search.py: 1 worker versus 8, on the same proof

What will you do? You will run `modeller/portfolio_search.py`, which solves the very same order-9 Golomb-ruler proof as the previous step, once with `num_search_workers=1` and once with `num_search_workers=8`, recording real wall-clock time and proof status for each.

Why? This is Week 3's own `num_search_workers` parameter, used there as an opaque performance knob — Unit 4 explains what it actually launches, and this step shows you a real, captured speed-up number rather than an assumed one.

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = seed
start = time.perf_counter()
status = solver.Solve(model)
wall_time = time.perf_counter() - start
```

```
speedup = single_time / multi_time if multi_time > 0 else float("inf")
print(f"num_search_workers=1 wall_time={single_time:.4f}s "
    f"length={single_length} status={single_status}")
print(f"num_search_workers={workers} wall_time={multi_time:.4f}s "
    f"length={multi_length} status={multi_status}")
print(f"speed-up (1-worker time / {workers}-worker time): {speedup:.2f}x")
```

Step 7 (continued)

Terminal

```
$ python modeller/portfolio_search.py
```

Expected result: Both runs report the same proven-optimal length (44) and status `OPTIMAL`; the 8-worker run is faster, but the reported speed-up is well under 8x. Your own numbers will differ from those below — report whatever your own run actually measures.

Expected Output

```
num_search_workers=1        wall_time=0.1707s  length=44  status=OPTIMAL
num_search_workers=8        wall_time=0.0443s  length=44  status=OPTIMAL
speed-up (1-worker time / 8-worker time): 3.86x
```

Tip

The proven length (44) must match exactly between the two runs — parallel search only changes how long the proof takes, never the answer it proves (Unit 4). If the two lengths ever disagree, something is wrong with the model, not with parallelism.

Step 8: Re-run it twice more and watch the ratio move

What will you do? Run the exact same command two more times in a row, and line up all three of your speed-up ratios — this step's two, plus the previous step's one — side by side.

Why? Unit 4's own reproducibility caveat states plainly that a multi-worker run is not guaranteed to reproduce the same timing across runs, even with a fixed `random_seed` — this step checks that claim on your own machine with three real data points instead of taking the chapter's word for it, and is exactly the evidence the chapter's own practice question about 1-versus-8-worker speed-up asks you to reason from.

Terminal

```
$ python modeller/portfolio_search.py
$ python modeller/portfolio_search.py
```

Expected result: The proven length stays 44 in every run, but the wall-clock times and the resulting speed-up ratio move around from run to run — sometimes closer to 8x, usually clearly less.

Expected Output

```
num_search_workers=1        wall_time=0.1753s  length=44  status=OPTIMAL
num_search_workers=8        wall_time=0.0518s  length=44  status=OPTIMAL
speed-up (1-worker time / 8-worker time): 3.38x
```

Expected Output

```
num_search_workers=1        wall_time=0.1782s  length=44  status=OPTIMAL
num_search_workers=8        wall_time=0.0654s  length=44  status=OPTIMAL
speed-up (1-worker time / 8-worker time): 2.72x
```

Checkpoint — verify before moving on

Three real runs, three different speed-up ratios (3.86x, 3.38x and 2.72x on this machine), all clearly below 8x, all proving the exact same optimal length. You have now both configured `num_search_workers` yourself and seen, on your own machine, why Unit 4 refuses to promise a specific speed-up number in advance.

6 Capstone — Combine Two Techniques on a Harder Instance

Step 9: Target the destroy rule on a 200-queen repair

What will you do? Run `lab/large_nqueens_lns_capstone.py` exactly as given — `N=200`, four times larger than `large_nqueens_lns.py`'s own reference instance, still using that script's plain random destroy rule — and record the final conflict count after 25 rounds. Then open the file, and where the `TODO` in `lns_destroy` says to, replace the random `rng.sample` line with a targeted rule: call `conflicted_rows(columns)` first, shuffle it, and take up to `k` rows from the front of that shuffled list, topping the neighbourhood up with random extra rows only if fewer than `k` rows are actually in conflict. Leave `REPAIR_WORKERS` at `1` for now, and re-run.

Why? This is Unit 2's own "choosing a neighbourhood" idea, tested on an instance neither of this week's own reference scripts has to face: a purely random destroy rule spends some of every round's repair effort on rows that were never in conflict to begin with; a targeted rule spends it where a fixed 25-round budget can actually make a difference.

```
Terminal
$ python lab/large_nqueens_lns_capstone.py
```

Expected result: Before your edit: real progress (224 conflicts down to 21) but a stall well short of zero, comfortably inside the 5-second time budget. After switching to the targeted destroy rule, the same 25 rounds reach 11 conflicts instead of 21 — a genuine improvement from choosing which rows to relax, not from running any longer.

Step 9 (continued)

Expected Output

```
Round 0: 224 conflicts
Round 1: 211 conflicts
Round 2: 185 conflicts
Round 3: 170 conflicts
Round 4: 154 conflicts
Round 5: 134 conflicts
Round 6: 122 conflicts
Round 7: 105 conflicts
Round 8: 98 conflicts
Round 9: 91 conflicts
Round 10: 80 conflicts
Round 11: 72 conflicts
Round 12: 67 conflicts
Round 13: 61 conflicts
Round 14: 58 conflicts
Round 15: 54 conflicts
Round 16: 51 conflicts
Round 17: 46 conflicts
Round 18: 38 conflicts
Round 19: 33 conflicts
Round 20: 28 conflicts
Round 21: 26 conflicts
Round 22: 25 conflicts
Round 23: 22 conflicts
Round 24: 22 conflicts
Round 25: 21 conflicts
Elapsed: 0.90s (budget 5.0s), final conflicts=21
```

Step 9 (continued)

Expected Output

```
Round 0: 224 conflicts
Round 1: 203 conflicts
Round 2: 184 conflicts
Round 3: 165 conflicts
Round 4: 143 conflicts
Round 5: 123 conflicts
Round 6: 106 conflicts
Round 7: 87 conflicts
Round 8: 65 conflicts
Round 9: 51 conflicts
Round 10: 35 conflicts
Round 11: 24 conflicts
Round 12: 24 conflicts
Round 13: 24 conflicts
Round 14: 24 conflicts
Round 15: 24 conflicts
Round 16: 24 conflicts
Round 17: 24 conflicts
Round 18: 24 conflicts
Round 19: 11 conflicts
Round 20: 11 conflicts
Round 21: 11 conflicts
Round 22: 11 conflicts
Round 23: 11 conflicts
Round 24: 11 conflicts
Round 25: 11 conflicts
Elapsed: 1.02s (budget 5.0s), final conflicts=11
```

Step 10: Add repair-step workers, and check whether they actually help here

What will you do? With the targeted destroy rule from the previous step still in place, complete the second `TODO`: raise `REPAIR_WORKERS` from 1 to 4, combining this week's Unit 2 and Unit 4 techniques in one script. Re-run once more.

Why? This is the week's integrative step (Objective H5): you have now selected and justified a combination of two of this week's techniques on an instance harder than any single reference script solves comfortably, and this step verifies the real result rather than assuming a combination of techniques must automatically help.

Terminal

```
$ python lab/large_nqueens_lns_capstone.py
```

Expected result: The same 21-to-11 improvement as the previous step's targeted run — `REPAIR_WORKERS=4` produces the identical round-by-round trace and the identical final conflict count (11); only the elapsed time changes slightly, still comfortably inside the 5-second budget.

Expected Output

```
Elapsed: 0.92s (budget 5.0s), final conflicts=11
```

Step 10 (continued)

Caution

It is tempting to read this as "portfolio search does not work here" — that is not what this result shows. Each repair step's sub-problem is only `k=10` relaxed rows against a fixed background; CP-SAT already finds that sub-problem's best repair in a few milliseconds with one worker, so four workers have nothing left to speed up. This is Unit 4's own diminishing-returns point — not every worker's job is to speed up the specific answer you are waiting for — now confirmed on your own real run rather than merely read in the chapter: the targeted destroy rule (Unit 2) is what earned this instance's real improvement, and the honest, real answer for `REPAIR_WORKERS` on *this particular* sub-problem size is "no measurable difference." Report that finding rather than the improvement you might have expected going in.

Checkpoint — verify before moving on

You selected a combination of two of this week's techniques for a harder instance than any single reference script here solves comfortably, implemented both in real code, and verified the real result within a stated time budget — and, just as importantly, correctly attributed which of the two techniques actually produced the improvement you observed. This is Objective H5 in full: not "combine techniques and hope," but "combine, verify, and report honestly what the numbers actually show."

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` from inside `kod_ornekleri`, then try the command again
`python modeller/min_conflicts.py` prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta04` before running any script
The `grep` command for Unit 3's restart/conflict lines prints nothing at all	`solver_log_restarts.py`'s output was not redirected into `ciktilar/solver-log-restarts.txt` before `grep` ran	Re-run the two-command sequence exactly as given, keeping the `>` redirection on the first line
`golomb_ruler.py`'s LNS-improve log stays at length 44 through all 15 rounds, never reaching 41	`NEIGHBOURHOOD_SIZE` was changed from 3 to 2	Restore it to 3 — an exhaustive check confirmed no 2-mark neighbourhood of this greedy start can improve it (see the `dikkat` box above)
The capstone script raises a `ValueError` about sampling more rows than are available	The targeted `lns_destroy`'s top-up step samples from a list built with the wrong count	Re-check the top-up slicing against this step's own description: build `remaining` from every row not already in `relaxed`, then take only `k - len(relaxed)` of it
`pytest -q` reports failures unrelated to your own edits	`modeller/*.py` files were edited directly instead of the `lab/` starter file	Restore `modeller/*.py` from the course repository; only `lab/large_nqueens_lns_capstone.py` is meant to be edited this week

5 • Follow-through **Transfer — Where Else?**

None of today's four mechanisms retires after this week. Week 5 treats today's restarts, nogood learning and local search as one half of the answer to "why is CP-SAT usually fast" — typical-case performance — and adds structural results (bounded treewidth, symmetry breaking) as the other half, for the worst-case questions this week's techniques cannot answer on their own. Week 12's rostering and Week 14's routing both lean on large neighbourhood search exactly as you ran it today, re-planning a large schedule after a small change rather than re-solving it from scratch, and Week 15's solver benchmarking is portfolio thinking applied one level up: comparing whole solvers and models against each other, not just search strategies inside one of them. Whatever hard instance you meet next, in a paper, in your thesis or in a future job, the four questions this sheet just walked you through are the ones to ask first: can you start from a complete guess and repair it, can you relax and re-solve a chunk of it with a real solver, is your search remembering its own failures, and would running several strategies at once, on separate cores, actually help — or, as the capstone's own honest result just showed you, would it not.